



MASTER OF CYBER SECURITY

Project Report – Cloud Intrusion Detection and Automated Response Using AWS VPC Flow Logs

SIT716 Computer Networks and Security

6.2HD Plus: Project Delivery

Student Name	Shloka Hitesh Kamdar
Student ID	226463713

Table of Contents

Table of Contents.....	2
1. Introduction	3
2. Architecture	3
3. Methodology.....	4
3.1 Baseline and Feature Engineering	4
3.2 Statistical Detection (Z-Score).....	5
3.3 Rule-Based Detection.....	5
3.4 Automated Response.....	6
3.5 Deviation from the Original Proposal	6
3.6 Multi-Source Distributed Testing Setup.....	7
4. Testing and Results	8
4.1 Test Design.....	8
4.2 Detection Metrics	8
4.3 Rule-Based vs Z-Score Comparison.....	11
4.4 Protocol-Level Evidence.....	12
5. Limitations.....	14
5.1 Single-VPC Boundary:.....	14
5.2 Allow-Only Model:	14
5.3 Delivery Ingestion Lag:	15
5.4 L3/L4 Telemetry Only:.....	15
6. Future Work	16
7. Conclusion	16
Appendix	17

1. Introduction

This report covers the design, implementation, and evaluation of a cloud-based intrusion detection system with an automated response mechanism, built entirely on AWS. The system uses VPC Flow Logs as its main source of network data and combines two detection approaches: a rule-based engine and a statistical anomaly detector. These are used to identify behaviours such as port scanning, SSH brute-force attempts, and ICMP flooding. When a threat is detected, the system automatically responds by adding a temporary Security Group rule through AWS Lambda, which is designed to expire after 15 minutes.

The system was tested using a distributed attack simulation involving two separate attacker instances. Instead of relying on expected results, the detection accuracy, false positive rate, and response time were measured using the data collected during the actual experiments. The testing also revealed some limitations, including an AWS constraint related to how Security Group rules work. These limitations have been included in the report rather than being overlooked, as they provide a more realistic view of how the system performs in an actual cloud environment.

2. Architecture

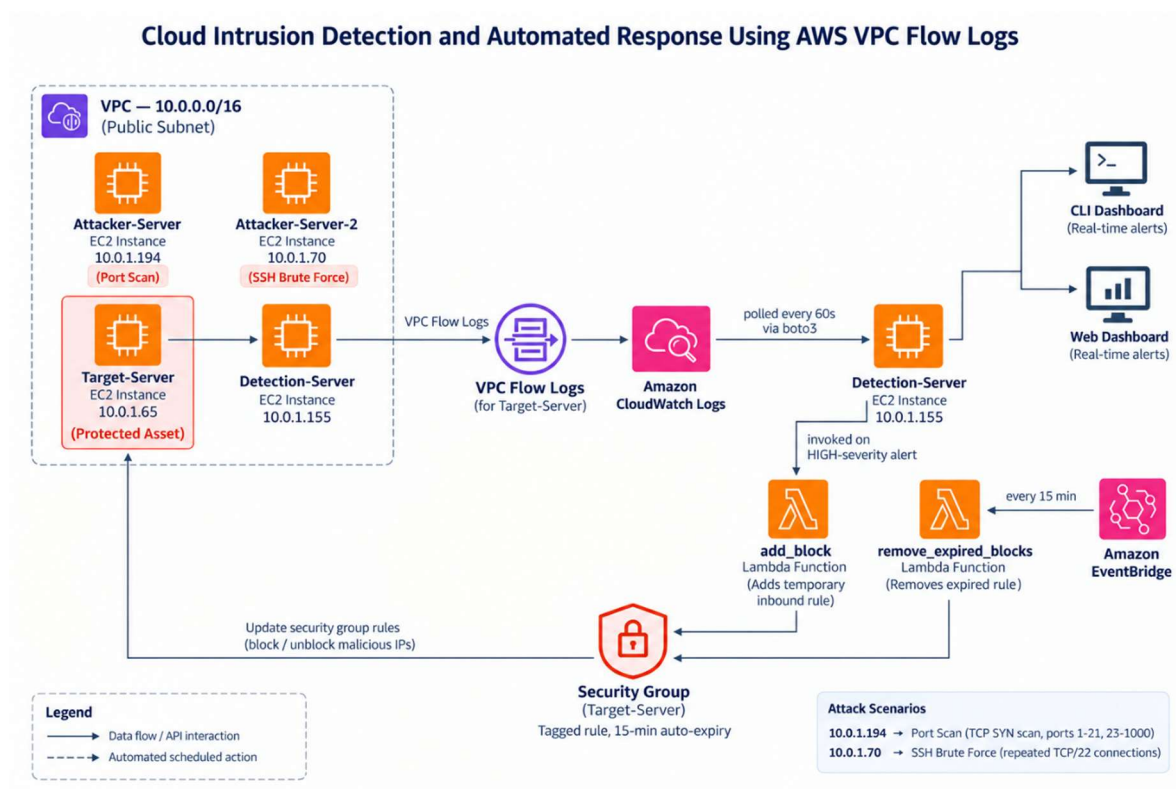


Figure 1 - System architecture: traffic flow from attack simulation, through detection, to automated containment and dual-dashboard visibility

The setup consists of four EC2 instances inside a custom VPC (10.0.0.0/16) with a public subnet. The four instances are the Target-Server, which acts as the protected system, two separate attacker instances (Attacker-Server and Attacker-Server-2) used to simulate distributed attacks, and the Detection-Server, which runs the detection and response pipeline every 60 seconds using a cron job.

VPC Flow Logs are enabled at the VPC level and send network traffic information directly to a CloudWatch Logs group. A dedicated IAM role is used to allow the Flow Logs service to write to this log group. The Detection-Server has its own IAM role, which gives it permission to read the CloudWatch Logs and trigger the automated-response Lambda function.

The automated response is handled by two Lambda functions. Both functions use IAM roles that allow them to read the required CloudWatch data and make changes to the Security Group. The `add_block` function is triggered by the detection engine when a high-confidence attack is detected. The second function, `remove_expired_blocks`, runs every 15 minutes using an EventBridge schedule and removes any blocking rules whose 15-minute expiry time has passed

Two dashboards consume the same underlying data: a CLI dashboard and a browser-based web dashboard, both reading live from `alerts.jsonl` and directly querying the Target's Security Group via `boto3`, so the displayed “active blocks” state is never a separate, potentially stale record — the Security Group itself is the single source of truth

3. Methodology

3.1 Baseline and Feature Engineering

Before running any attack simulations, a baseline of normal network traffic was collected over a 45–60-minute period. A Python pipeline consisting of `fetcher.py`, `parser.py`, and `aggregator.py` was used to collect the VPC Flow Log records through `boto3`, parse the version 2 flow log fields, and group the data into 60-second windows based on the source IP.

A 60-second window was selected to provide a balance between quick detection and reliable data. Using a shorter window could result in unstable or noisy features because there may not be enough traffic in each window. On the other hand, a longer window would slow down detection and could combine separate network events into the same window.

For each window, nine behavioural features are calculated. These include `total_connections` and `connections_per_second`, which represent the overall traffic volume and connection rate, and `unique_dst_ports`, which shows how many different destination ports were contacted and is particularly useful for identifying port scanning. `avg_bytes_per_conn` and `avg_packets_per_conn` provide information about the amount of data transferred per connection and help distinguish normal data transfer from probing activity. `reject_ratio` measures the proportion of rejected connections and can help identify scanning or blocked brute-force attempts. The system also records `total_packets` and `total_bytes` to measure overall traffic volume, along with `avg_duration_seconds`, which represents the average session length. Very short or near-zero sessions can be a sign of scanning or probing rather than normal interactive traffic.

3.2 Statistical Detection (Z-Score)

For each feature, the mean and standard deviation are first calculated using the baseline traffic. Four of the volume-based features (`avg_bytes_per_conn`, `avg_packets_per_conn`, `total_bytes`, and `total_packets`) tend to have a strong right-skew, so $\log_{10}$ transformation is applied to them before calculating these values. This helps reduce the effect of occasional but legitimate high-volume activities, such as an apt update, an scp file transfer, or a large curl download. Without this transformation, these events can increase the standard deviation too much and make normal traffic appear less unusual than it is.

For each new 60-second window, a z-score is calculated for every feature by comparing the observed value with the baseline mean and standard deviation.

A minimum standard deviation of 0.01 is used to avoid division-by-zero errors when a feature has very little or no variation in the baseline data. The highest z-score across all features is then compared against a threshold of 4.0σ . If the maximum z-score is above this threshold, the window is flagged as a MEDIUM-severity statistical anomaly.

The threshold was initially set to 3.0σ but was increased to 4.0σ after the log transformation was introduced. The transformation reduced the spread of the baseline data, which meant that a 3σ threshold was triggering too often for normal variations in traffic. The 4σ threshold was therefore used to reduce these false alerts while still identifying significant deviations from the baseline.

3.3 Rule-Based Detection

Alongside the statistical detection layer, three rule-based signatures are used to identify specific attack patterns. These rules are given HIGH severity with a confidence of 1.0 because they are based on clearly defined conditions rather than a statistical prediction.

- **PORT_SCAN:** This rule triggers when `unique_dst_ports` > 40, `avg_bytes_per_conn` < 100, and `reject_ratio` > 0.8. A port scan typically attempts connections to many different ports while sending very little data. A high rejection rate also helps distinguish this behaviour from normal network activity.
- **SSH_BRUTE_FORCE:** This rule triggers when `port_22_count` > 0, `total_connections` > 100, and `avg_bytes_per_conn` < 150. The idea is to detect many small connection attempts focused on SSH (port 22) within the same 60-second window, which is consistent with a brute-force attack.
- **ICMP_FLOOD:** This rule triggers when the primary protocol is ICMP and `total_packets` > 500. A sudden and unusually high number of ICMP packets within a short period is treated as a potential ICMP flooding attack.

3.4 Automated Response

When a HIGH-severity rule is triggered, the detection engine calls the `add_block` Lambda using `boto3` and waits for its response to confirm the block succeeded before continuing. It passes the attacker's IP address, the reason for the block, and a 15-minute duration. Before adding the block, the Lambda checks a whitelist to make sure important addresses are not accidentally blocked. This includes the management IP and the AWS metadata service address.

If the IP is not whitelisted, the Lambda adds an inbound rule to the Target-Server's Security Group using `AuthorizeSecurityGroupIngress`. The rule description stores the block time, expiry time, and reason in a structured format. During implementation, the original `|` delimiter was found to be outside the characters allowed in AWS Security Group descriptions, so it was replaced with `;` and `=` before deployment.

A second Lambda function, `remove_expired_blocks`, handles the removal of expired rules. It runs independently every 15 minutes through an EventBridge schedule. Instead of relying on a separate database, it checks the Target-Server's Security Group directly, looks for the auto-blocked rules, and removes any rule whose expiry time has passed. This means the Security Group itself acts as the source of truth, avoiding the possibility of a separate database becoming out of sync.

Only HIGH-severity rule-based detections can trigger an automatic block. MEDIUM-severity statistical anomalies are logged and reported but do not result in an automatic response. This was intentionally designed to keep automated actions limited to detections that meet specific, deterministic conditions and reduce the chance of blocking legitimate traffic because of a false positive.

A 5-minute deduplication cooldown is also applied per IP and severity. This prevents the same ongoing attack from generating repeated alerts and blocks within a short period. However, if a later detection has a higher severity than the one already recorded, it is allowed through immediately rather than being suppressed by the cooldown.

3.5 Deviation from the Original Proposal

The original proposal included a Random Forest classifier as part of the detection engine and used the title “AI-Assisted Cloud Intrusion Detection”. During the proposal review, feedback highlighted three main areas that needed more work. The machine learning component needed clearer details about the source of labelled training data, feature selection, why Random Forest was being used, and how the model would be validated. The automated response also needed to explain how Security Group changes would be triggered, verified, and rolled back if a false positive occurred. Finally, the overall scope was considered quite ambitious for a unit project.

These points were addressed during implementation. Instead of using a machine learning classifier, the final system uses the rule-based and statistical detection layers described earlier. This removed the need for labelled training data and model validation, while also making the detection process easier to interpret. Each alert can be traced back to the specific rule or statistical feature that caused it, which supports the requirement to explain why an event was detected.

The automated response concerns were also addressed through the design described in Section 3.4. Security Group blocks are applied through Lambda, given a defined expiry time, and their status can be checked directly through the system dashboards. Rather than relying on manual rollback, expired blocks are automatically removed after the set time.

Removing the machine learning component also allowed more time to be spent on the automated response pipeline and the evaluation of the system using actual attack data. As a result, the project scope became more focused and achievable while still demonstrating the core objectives of cloud-based intrusion detection and automated response.

The project was therefore renamed “Cloud Intrusion Detection and Automated Response Using AWS VPC Flow Logs” to accurately reflect the system that was ultimately implemented, since the final version does not use machine learning

3.6 Multi-Source Distributed Testing Setup

o check whether the detection and containment system works correctly when attacks come from more than one source at the same time, two separate attacker instances were set up within the same VPC. This allowed the system to be tested against concurrent, multi-source attacks rather than only a single attacker.

Each attacker instance runs a dedicated, single-purpose attack script, a port scan on one instance and an SSH brute-force burst on the other, so that detection and per-IP containment could be validated against two concurrent, distinct attack types running from two different sources at the same time, rather than the same repeated pattern from one source. Each script automatically detects its own private IP address and prints a timestamp snippet marking when its attack phase started and ended. These timestamps from both attackers are combined into a single ground-truth file and used to calculate the detection and response metrics, making it easier to match the generated attacks with the events recorded by the detection system.

4. Testing and Results

4.1 Test Design

During development, a combined mixed-traffic test script was used to validate the pipeline end-to-end: 10 minutes of normal traffic using ping and curl to the Target-Server, an attack phase that first performs an nmap port scan across ports 1–1000 and then runs an SSH brute-force simulation, and then a further 5 minutes of normal traffic.

During this development testing, it became clear that using Hydra by itself was not enough to trigger the SSH_BRUTE_FORCE rule. Hydra was only making five password attempts, and password authentication was disabled on the Target-Server, so the number of connections never reached the rule's threshold of 100 connections.

To make the test reliably generate the required traffic, the script was extended with a 60-second burst of high-volume connection attempts to port 22. Each attempt uses an explicit 0.3-second timeout. This was added after testing showed that a connection attempt without a timeout could hang indefinitely when the Target-Server silently dropped packets instead of actively rejecting them. This change ensured that the test generated enough SSH connection traffic for the detection rule to be evaluated consistently.

For the formal multi-attacker test reported in Section 4.2, this combined script was split into two dedicated scripts, one per attacker instance, so that the port scan and the SSH brute-force burst could be generated concurrently from two separate sources rather than sequentially from one, as described in Section 3.6.

4.2 Detection Metrics

The formal multi-attacker test was run with both Attacker instances launching their attack phase within seconds of each other. The detection engine's raw per-window decision log was compared against the recorded ground-truth attack windows for both attackers:

Metric	Value
True Positives	15
False Positives	92
True Negatives	454
False Negatives	0
Detection Rate	1.000

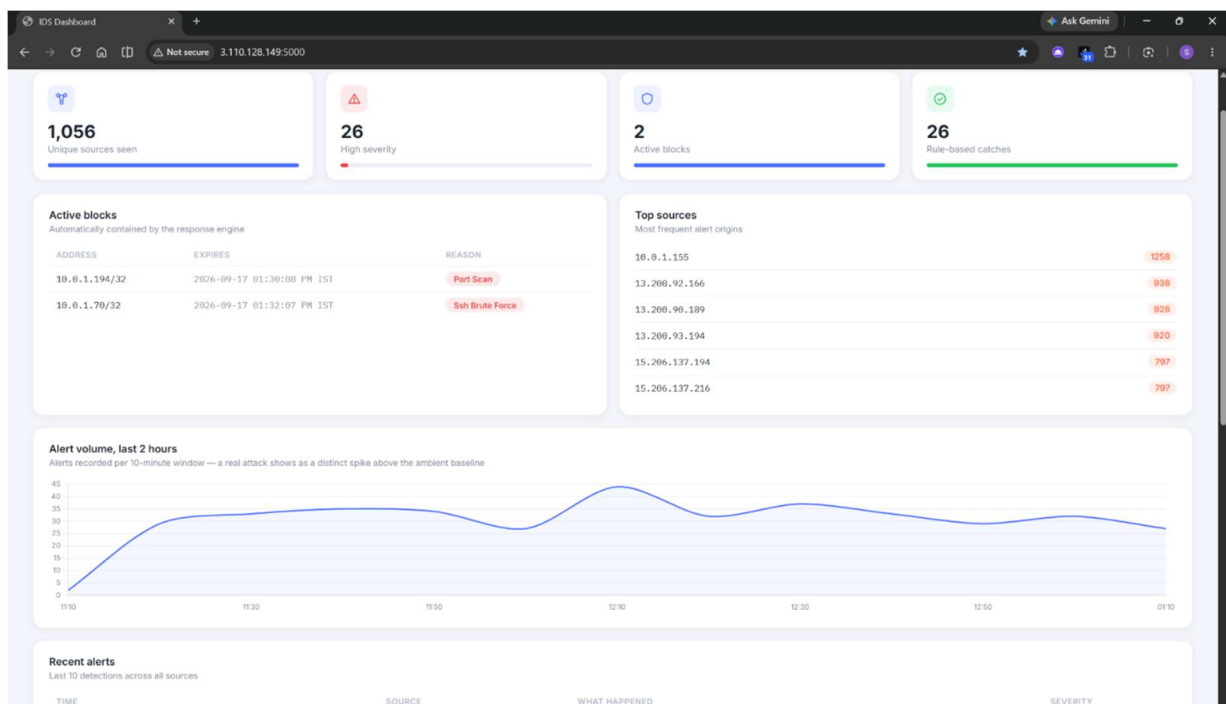


Figure 3 - Web dashboard during the active block window: both attacker IPs auto-blocked, one tagged Port Scan and one tagged Ssh Brute Force

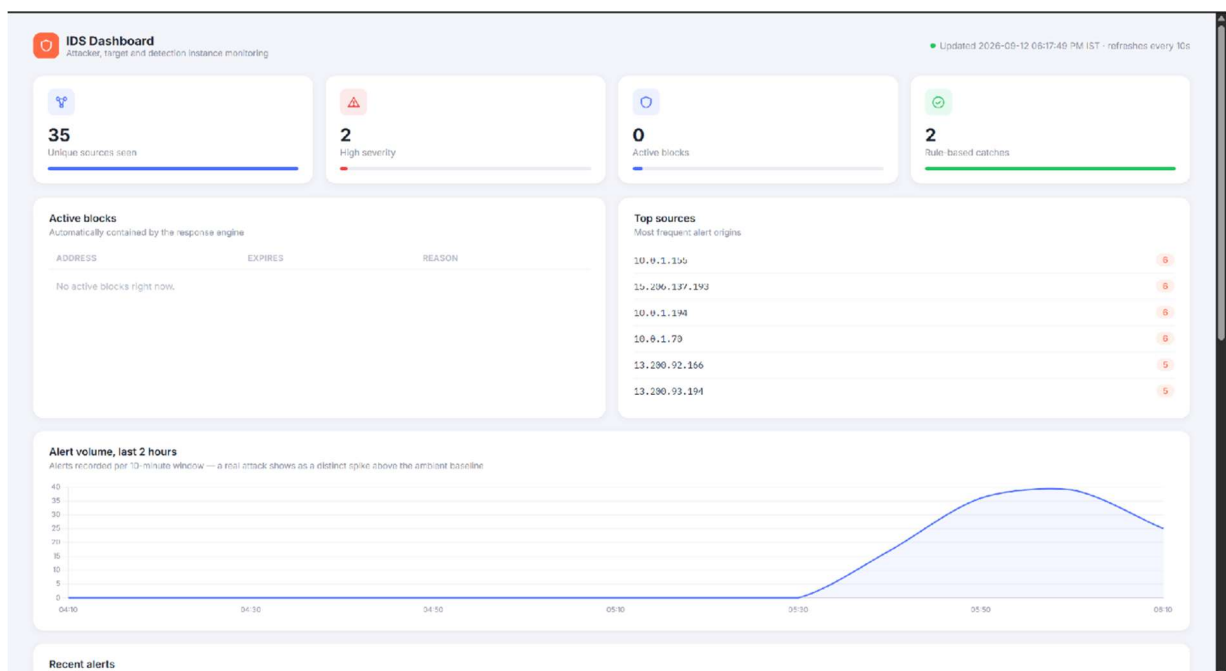


Figure 4 - The same dashboard after automatic expiry: both blocks cleared with no manual intervention

4.3 Rule-Based vs Z-Score Comparison

Testing surfaced concrete, complementary evidence for both detection layers, rather than one clearly outperforming the other:

Scenario	Rule-Based Outcome	Z-Score Outcome
Concentrated high-volume connection burst	Fired reliably (HIGH), triggered auto-block	Also fired (MEDIUM) in the same window
Port scan spanning more than one 60s window	Sometimes missed — <code>unique_dst_ports</code> diluted below 40 once scan traffic split across windows	Same window-boundary limitation applies, though ambient training data still occasionally flagged unrelated features
Attack with <code>reject_ratio</code> = 0.50 (below the 0.8 rule threshold)	Did not fire	Fired at $z = 1098$ — correctly caught what the rule missed entirely
Ambient internet background scanning noise	Never fires — does not match any signature	Fires very frequently — documented limitation, Section 5

These results show the two layers are genuinely complementary rather than redundant. The deterministic rules give zero-ambiguity, high-confidence triggers for automated action, but are brittle to attacks that do not fit their exact shape or that happen to straddle a 60-second window boundary. The statistical layer is far more sensitive and catches deviations the rules miss entirely, at the cost of firing on a large volume of benign ambient noise — the reason it was deliberately excluded from triggering automated blocking (Section 3.4).

4.4 Protocol-Level Evidence

Packet captures taken on the Target during the formal test provide direct protocol-level confirmation of the behavioural patterns the feature-based detection relies on.

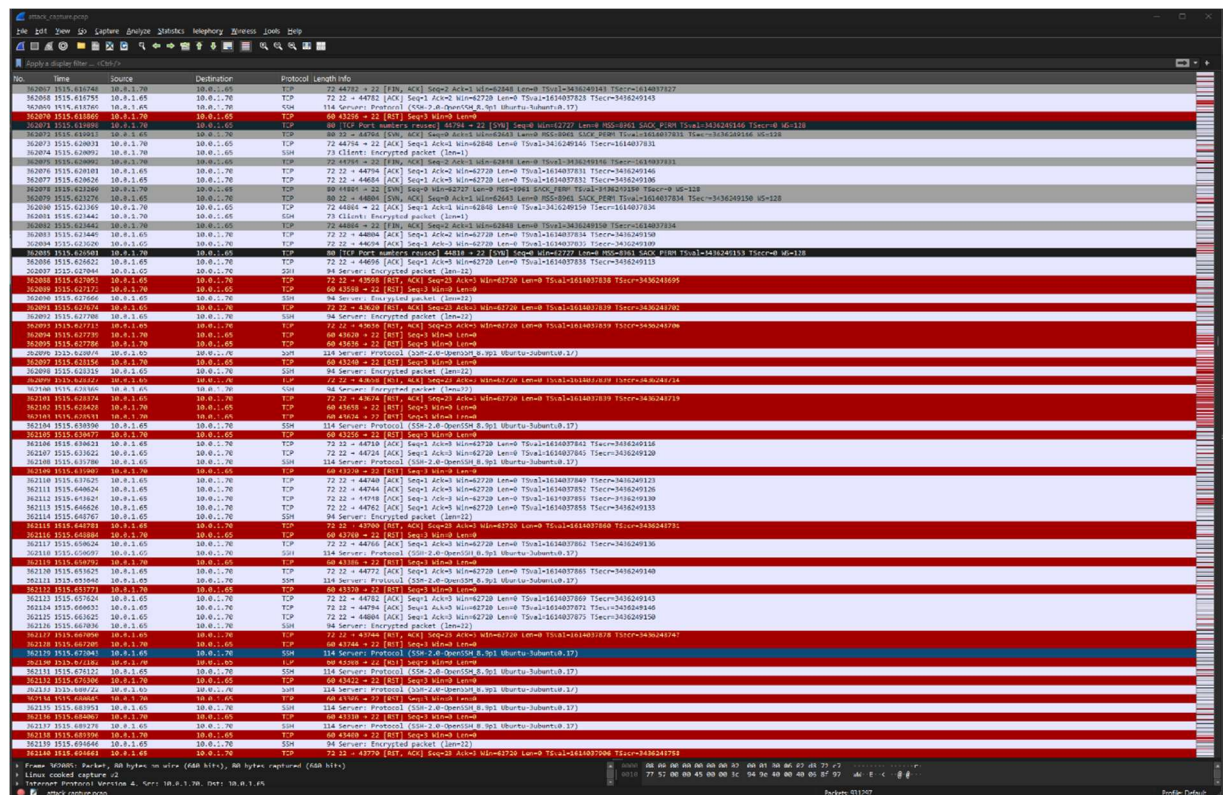


Figure 5 - Normal / baseline traffic captured on the Target before the attack phase: legitimate protocol exchanges consistent with the profile the baseline statistics were built from

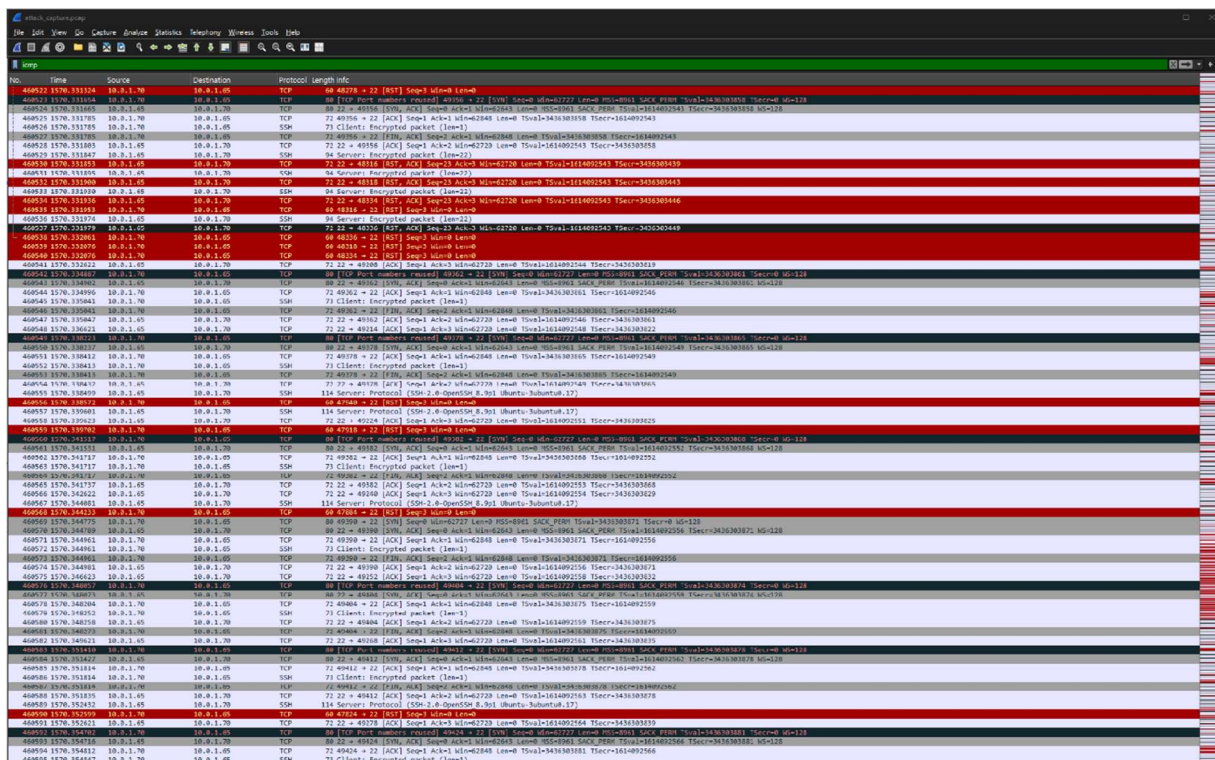


Figure 6 - Port scan traffic: a rapid fan-out of SYN packets across many distinct destination ports, the defining packet-level signature of a port scan

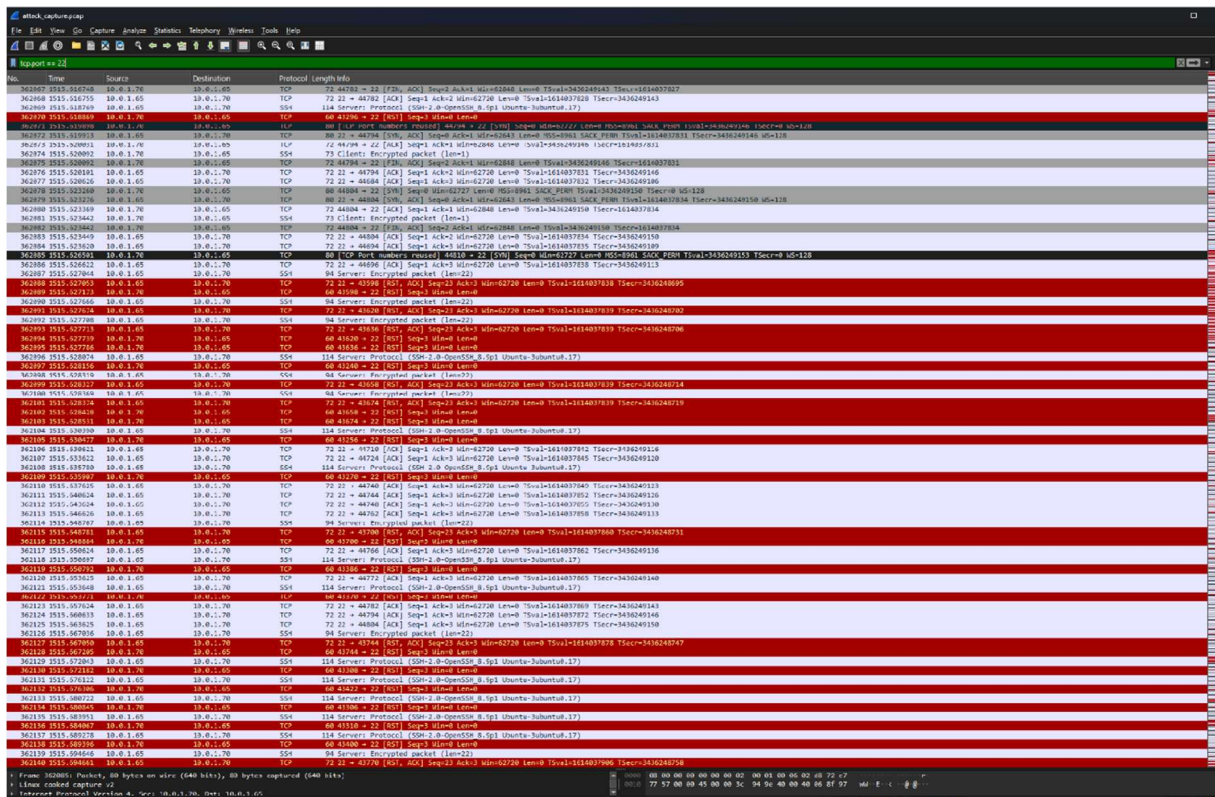


Figure 7 - SSH-specific traffic showing genuine OpenSSH protocol banner exchanges alongside rapid connect/FIN/RST cycles concentrated on port 22, contrasting with the port scan's fan-out pattern

5. Limitations

The following limitations were identified during the development and testing of the system. These are based on what was observed during testing rather than assumptions about how the system would behave.

5.1 Single-VPC Boundary:

The detection and response system operates within a single VPC and does not correlate events across multiple AWS accounts or VPCs. Flow log collection, the baseline statistics, and the Security Group that gets updated are all scoped to the Attacker, Target, and Detection instances inside the same 10.0.0.0/16 VPC described in Section 2. An attacker distributing traffic across separate VPCs or accounts would not be visible to this system as a single correlated campaign — each VPC would need its own flow log pipeline and baseline. This was an accepted scope decision for a unit project rather than a technical limitation of the detection approach itself, and is addressed as a possible extension in Section 6.

5.2 Allow-Only Model:

Another important limitation was related to the AWS Security Group mechanism used for automated response. Security Groups do not support explicit DENY rules. They work using allow rules, with traffic that does not match an allowed rule being denied by default.

The `add_block` Lambda correctly performs the detection → action → tagging → expiry workflow. However, the rule added for a detected attacker technically allows traffic from that IP rather than explicitly denying it. This was confirmed during testing. While an IP was shown as blocked in the dashboard and through `describe-security-groups`, a ping from the supposedly blocked attacker still reached the Target-Server, with 0% packet loss across 77 packets and consistent sub-millisecond latency. A curl request from another blocked IP also returned an immediate "Connection refused" response instead of timing out. This showed that the traffic was reaching the Target-Server's operating system rather than being dropped by the network layer.

For a production system that requires actual deny-based containment, a Network ACL or a host-based solution such as iptables triggered through AWS Systems Manager would be more suitable. However, these approaches would add additional complexity. For example, NACL entries do not provide a free-text description field, so the current approach of storing the block time, expiry, and reason directly in the Security Group rule would need to be replaced with a separate state-management mechanism. This approach was considered during the project but was not implemented due to the project timeline. Therefore, this is treated as an architectural limitation of the selected AWS mechanism rather than a failure of the automation pipeline itself. The complete detection → decision → action → automatic expiry workflow operates as intended, even though the chosen Security Group mechanism does not provide true network-level isolation.

5.3 Delivery Ingestion Lag:

One of the main limitations observed was the delay in VPC Flow Log delivery to CloudWatch. Across different test runs, logs were sometimes available in under a minute, while in other cases delivery took around 15–20 minutes. The response time reported in Section 4.2 comes from a run where the logs were delivered relatively quickly. This means the overall response time should be considered as two separate parts: the time AWS takes to deliver the flow logs, which is outside the detection engine's control, and the time taken by the detection engine itself. Once the data is available, the detection and processing take less than a second.

The fixed 60-second aggregation window adds a related limitation: if an attack continues across more than one window, its traffic can be split between those windows. This may reduce `unique_dst_ports` enough that the `PORT_SCAN` threshold is not reached in any individual window, even though a scan has clearly taken place. This behaviour was observed during testing. In some runs, only `PORT_SCAN` was triggered, while in others, including the formal multi-attacker test in Section 4.2, only `SSH_BRUTE_FORCE` was triggered. The result depended on the duration of the scan and how the SSH connection burst aligned with the 60-second window boundaries.

5.4 L3/L4 Telemetry Only:

The system performs detection out-of-band using flow log records sent to CloudWatch rather than inspecting packets directly while they are travelling through the network. This means that by the time an attack is detected, the traffic that caused the detection has already occurred. This is like the general approach used by AWS GuardDuty, where detection is based on collected data and remediation can be triggered separately. A true inline IPS would require a different architecture, such as AWS Network Firewall or a Gateway Load Balancer appliance, which would add considerably more infrastructure and complexity.

A related consequence is that VPC Flow Logs only ever provide Layer 3/4 telemetry — source and destination IP, port, protocol, packet and byte counts — with no packet payload or application-layer content. This is why the system's detection rules and statistical features are built entirely from connection metadata rather than inspecting what the traffic actually contains, and why HTTP-level attacks such as SQL injection or cross-site scripting fall outside this system's scope entirely, regardless of volume or timing. A Web Application Firewall or similar payload-aware layer would be needed to address that gap, as noted in Section 6.

6. Future Work

There are several areas where the system could be extended in the future:

- **Machine learning classifier:** The original proposal included a Random Forest classifier, which was removed because there was not enough labelled training data at the time. The attack simulations carried out for this project have now generated real labelled data, so a machine learning approach could be explored in future work. The existing features and collected attack data could be used to train and evaluate the classifier.
- **Genuine deny-based containment:** The current automated response uses Security Group rules, which do not provide a true deny mechanism. A future version could use Network ACLs or an SSM-triggered iptables mechanism to provide actual blocking at the network or host level. A lightweight state store could also be introduced to keep track of block times, expiry, and reasons.
- **Multi-account and multi-region support:** The current implementation is limited to a single VPC. It could be extended to collect and analyse flow logs from multiple AWS accounts and regions, making the system more suitable for larger cloud environments.
- **Web Application Firewall:** A WAF layer could be added to detect HTTP-level attacks such as SQL injection and cross-site scripting. These attacks are outside the current system because the project focuses mainly on network-level traffic.
- **Inline packet inspection:** Another possible extension would be to introduce AWS Network Firewall or a Gateway Load Balancer appliance for inline traffic inspection. This would allow threats to be detected and blocked closer to the point where the traffic enters the network, reducing the delay associated with the current log-based, out-of-band detection approach.

7. Conclusion

This project delivered a working, end-to-end cloud intrusion detection and automated response system, built and validated entirely on AWS infrastructure. The core detects → decide → respond → expire loop was demonstrated and measured under both single- and multi-source simulated attack, with a perfect detection rate and sub-50-second response time recorded in the formal test run reported in Section 4.

Where the delivered system diverges from the original proposal — most notably the replacement of a machine learning classifier with rule-based and statistical detection — the change is directly traceable to specific proposal feedback and is shown to strengthen, rather than weaken, the project's demonstration of the unit's learning outcomes. Genuine limitations, including an AWS platform constraint on Security Group semantics discovered and verified empirically during testing, are documented rather than omitted, reflecting the applied, evidence-based standard this report has aimed to meet throughout.

Appendix

Project Links

GitHub repository (Terraform, detection engine, Lambda functions, attack simulation scripts):

<https://github.com/shlokamdar/sit716-hdplus>

Presentation recording

<https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=5ba8d09f-de41-4ab1-a83e-b4c9009c1dc0>